

REST Assured API Automation Framework

1. Introduction

This project is a simple API automation framework developed using Java, Maven, TestNG and Rest Assured. The framework follows a clean structure by separating configuration, request specification, API client and test classes. This keeps the test code easy to read, maintain and extend.

The automation covers a simple API scenario where posts for a specific user are retrieved and validated.

2. Technologies Used

- Java 17
 - Maven
 - TestNG
 - Rest Assured
-

3. BaseTest

The BaseTest class is the parent class for all API test classes.

It initializes the Request Specification before test execution so that every test can reuse the same configuration.

Responsibilities

- Initialize RequestSpecification
- Make RequestSpecification available to child test classes

Sample Code

@BeforeClass

```
public void setUp() {  
    SpecFactory specFactory = new SpecFactory();  
    requestSpec = specFactory.createRequestSpec();  
}
```

4. ConfigManager

Instead of hardcoding the API URL, the framework reads it from the configuration file.

This class loads the properties file and provides methods to access configuration values.

Sample Code

```
public String getBaseUrl() {  
    return properties.getProperty("baseUrl");  
}
```

Configuration file:

```
baseUrl=https://jsonplaceholder.typicode.com
```

This makes changing environments simple.

5. SpecFactory

The SpecFactory class creates a reusable Request Specification used across all API requests.

Responsibilities

- Read Base URL
- Configure Content-Type
- Return RequestSpecification

Sample Code

```
public RequestSpecification createRequestSpec() {  
  
    return given()  
        .baseUrl(configManager.getBaseUrl())  
        .contentType(ContentType.JSON);  
}
```

6. PostClient

The PostClient class contains only API request methods.

It performs the HTTP request and returns the response without performing any validations.

GET Posts

```
public Response getPostsByUserId(int userId) {  
  
    return given()  
        .spec(requestSpec)  
        .queryParams("userId", userId)  
        .get("/posts");  
}
```

The client class is responsible only for making API calls.

7. Test Class

The test class contains only the execution flow and validations.

Business actions are delegated to the client class.

Test Flow

```
PostClient client = new PostClient(requestSpec);
```

```
Response response =
```

```
    client.getPostsByUserId(1);
```

Validations

- Verify Status Code = 200
- Verify Response is not empty
- Extract response using JsonPath
- Loop through every post
- Verify every userId equals 1

Example

```
response.then().statusCode(200);

JsonPath jsonPath = response.jsonPath();

List<Map<String, Object>> posts =
    jsonPath.getList("$");

for (Map<String, Object> post : posts) {

    Assert.assertEquals(post.get("userId"), 1);
}
```

8. Framework Design

The framework follows a simple layered structure.

- Configuration values are managed in ConfigManager.
- Request Specification is created in SpecFactory.
- API requests are written inside Client classes.
- Test classes contain only validations and execution flow.

This separation keeps the framework clean and reusable.

9. Rest Assured Features Used

The framework uses several Rest Assured features.

Examples

- RequestSpecification
- Query Parameters
- GET Request
- Response
- JsonPath

Example

```
.queryParams("userId", 1)
.get("/posts")

JsonPath jsonPath = response.jsonPath();
```

10. Overall Test Scenario

The automated scenario performs the following steps:

1. Initialize Request Specification.
 2. Read Base URL from configuration.
 3. Send GET request to /posts.
 4. Pass userId=1 as Query Parameter.
 5. Verify Status Code is 200.
 6. Verify response is not empty.
 7. Extract response using JsonPath.
 8. Loop through all posts.
 9. Verify every post contains userId = 1.
-

11. Conclusion

This project demonstrates a simple API automation framework using Rest Assured and TestNG. The framework separates configuration, request creation, API requests and test validations into different classes, making the code easy to maintain and extend. The completed test successfully validates the posts returned for a specific user using query parameters, JsonPath and assertions.